

论文模板 01 · 微服务架构设计

配套素材见 [00-项目素材库.md](#) 。本模板所有  内容均来自 BASE HOME 真实源码。

一、真题常见问法

变体 A（架构设计型）

请论述你参与的某系统的微服务架构设计，包括服务拆分原则、服务间通信方式、服务治理机制，并说明你在设计过程中遇到的挑战及解决方案。

变体 B（拆分决策型）

微服务架构的核心难点之一是服务粒度的划分。请结合你参与的项目，论述你如何确定服务边界，以及拆分过程中如何保证数据一致性。

变体 C（挑战型）

请论述微服务架构在实际落地中面临的挑战（如分布式事务、服务依赖、版本管理），并结合项目说明你的应对措施。

判断：题目若强调「拆分/边界」→ 重心放服务划分理由；强调「治理/挑战」→ 重心放契约一致性 + 版本治理 + 最终一致性。

二、摘要模板（约 300 字）

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台的微服务架构设计与实现。

该平台面向日本 H.I.S. 集团的营业本部与旅行者，覆盖预约管理（予約管理）、客户消息（顧客メッセージ）、通知发送（通知配信）与主数据联动等核心业务。

本人担任架构师，负责服务边界划分、服务间通信契约设计与构建治理体系。

针对【业务伸缩特征差异大】、【跨语言调用契约易漂移】、【新旧技术栈长期并存】三大挑战，

本文从【服务拆分】、【契约驱动】、【渐进演进】三个维度进行设计：

按业务特征与伸缩曲线将系统划分为 Web、Service、Batch、FaaS 等独立部署单元，

以 Protocol Buffers 作为唯一契约源驱动 Java、Go、PHP 三语言客户端生成，

并通过统一版本管理插件锁定关键依赖，规避了依赖漂移引发的运行时故障。

系统上线后【 填效果】，验证了方案的可行性。

三、正文五段框架

第 1 段 · 项目背景与本人角色 (约 400 字)

写什么：

- 项目名称、所属企业、业务定位、服务对象
- 项目规模（用素材库第二节的数字：8 个模块、17 个 RPC、79 个实体、14 个 Lambda）
- 本人角色与 3 项具体职责
- 一句话点出「为什么需要微服务」（引出后文）

本项目可直接用：

BASE HOME 是 H.I.S. 面向旅游业务建设的支撑平台，覆盖预约管理、客户消息、通知发送与主数据联动等业务域，采用 Java 21 + Spring Boot 3.5.7 技术栈，按多模块 Gradle 结构划分为 Web、Service、Batch、Common、Spec、FaaS 等模块，部署于 AWS 东京区域（ap-northeast-1）。本人担任架构师，负责服务边界划分、服务间通信契约设计以及构建版本治理。

第 2 段 · 架构设计 (约 800 字，本文核心)

必须先讲「拆分依据」——这是区分架构师与工程师的关键。

推荐三个拆分维度（本项目真实成立）：

维度一：按伸缩特征拆分

Web 层承载营业本部员工的交互操作，流量呈**工作日昼间高峰**特征；Batch 层承载成本文件生成、邮件发送等定时作业，呈**夜间定时峰值**且瞬时资源占用大；FaaS 层承载数据同步，呈**持续低频**特征。若合并部署，批处理的资源峰值会直接冲击在线预约接口的响应时间。因此我将三者拆分为独立部署单元，使各层可按自身曲线独立扩缩容。**权衡说明**：这牺牲了单体部署的调用简单性，换来了资源隔离与独立伸缩能力。

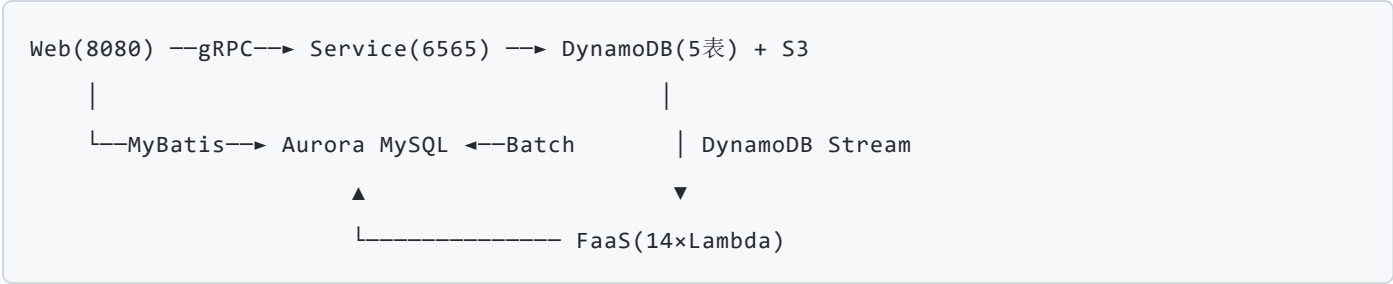
维度二：按访问协议与调用方拆分

内部服务间采用 **gRPC**（`base-home-service` 暴露 6565 端口，17 个 RPC），而非 REST。原因是服务间调用对性能与强类型契约更敏感，gRPC 基于 HTTP/2 多路复用且由 proto 保证强类型；对外则通过 Web 层的 REST 接口（`/api/v1/**`）暴露，兼顾浏览器友好性与前后端契约稳定性。

维度三：按数据存储特征拆分（本项目最有说服力的一点）

我采用**混合存储 (Polyglot Persistence)**：Aurora MySQL 承载强一致交易与复杂关联查询（79 张业务/主数据表）；DynamoDB 承载 5 张 schema-less 高弹性表（travel / document / parental / sequence / message_thread）；S3 承载文件实体。服务边界与数据边界对齐，避免多个服务共享同一张表造成的隐性耦合。

画架构图（文字版，考场上画方框箭头即可）：



第 3 段 • 关键技术实现 (约 700 字)

要素一：契约驱动的服务通信

系统需同时为 Java、Go、PHP 三种技术栈的调用方提供服务。我在 `base-home-spec` 模块中以 `protobuf` 作为**唯一契约源**，通过构建脚本编排 `build-java → build-go → build-php → generate-all`，分别生成三语言产物并独立发布（Maven / Go module / Composer）。同时用 `--openapiv2_out` 产出 OpenAPI 描述文件，经 MkDocs 发布为在线文档。该设计使「17 个 RPC 的接口口径」在三端天然一致，把接口不一致问题从「运行时联调发现」提前到「构建期暴露」。

要素二：服务端的分发与复用

`base-home-service` 采用单一入口 `BaseHomeService`（`@GRpcService`）承接 17 个 RPC，统一经 `execute(IGrpcService, req, respObserver)` 方法分发到各业务实现类，共 18 个实现类；跨业务域的能力（文档、消息线程、亲权同意书、序列号、旅行信息）下沉到 `service/shared/` 复用，避免同一逻辑在多个 RPC 中重复实现。

要素三：构建期依赖治理

项目通过独立的 `base-home-versions` Gradle 插件统一版本源（`base-home.versions.gradle`），各模块以 `apply plugin: 'base-home.versions'` 接入，并用 `resolutionStrategy { force ... }` 锁定关键组件。

第 4 段 • 难点与解决 (约 700 字)

从素材库第四节挑 2 ~ 3 个。微服务主题的最佳组合：

优先级	难点	为什么适合微服务主题
首选	① 一份 proto 契约驱动三语言	直接对应「服务间通信」考点
次选	② protobuf 版本冲突致启动失败	直接对应「服务治理」考点
三选	⑥ Serverless 数据同步（最终一致性）	直接对应「分布式数据一致性」考点

难点写法三段式（每个难点 200 字左右）：

问题 → 为什么这是个架构问题（不是编码问题）
方案 → 你的决策 + 为什么选它（对比其他选项）
效果 → ◆ 量化数字

示例（难点 ②）：

我们在升级 protobuf 依赖时，`proto-google-common-protos` 被解析到 2.64.0-rc1，导致 gRPC 服务启动直接失败。更棘手的是，由于版本声明采用了动态区间（`protobufVersion = '[3,4)'`、`grpcVersion = 'latest.release'`），该问题在构建期完全不显现，只在运行时暴露——这属于典型的依赖漂移风险。我的判断是：这不是一次偶发故障，而是「版本契约不明确」这一架构缺陷的必然结果。因此我没有只做局部修复，而是在统一版本源中将该组件强制锁定到 2.63.2，并同步锁定 MyBatis 3.0.5 等关键组件，把隐式版本变成显式契约。改造后构建具备可复现性，同一提交在任何机器上产物一致。

第 5 段 · 总结与不足（约 200 字）

目前 BASE HOME 已稳定支撑【◆ 业务量】，【◆ 核心指标】达到【◆ 数值】。
回顾建设过程，仍存在两点不足：
一是跨语言契约虽有统一来源，但破坏性变更尚未实现自动化的兼容性检测；
二是 AWS SDK v1 与 v2 仍处于并存状态，长期维护成本尚未完全消除。
后续计划通过引入契约兼容性校验工具、逐步收敛 SDK 版本进一步优化。
通过本项目，我深刻体会到微服务架构的价值不在于“拆”本身，
而在于让每个服务能够按自身的业务节奏独立演进。

四、本项目填充点速查

论文要素	直接可写的内容
服务划分单元	Web / Service / Batch / Common / Spec / FaaS
通信方式	服务间 gRPC (HTTP/2 + 强类型契约) ； 对外 REST <code>/api/v1/**</code>
RPC 数量	17 个 RPC，18 个服务端实现类
服务内复用	<code>service/shared/</code> 5 个共享服务
契约管理	protobuf 唯一契约源，Java/Go/PHP 三语言 + OpenAPI 文档
数据边界	Aurora MySQL (79 表) + DynamoDB (5 表) + S3
版本治理	<code>base-home-versions</code> 插件 + <code>resolutionStrategy force</code>
独立部署	每个 Lambda 独立 Dockerfile + 独立 GitHub Actions
环境隔离	DynamoDB 表名带环境前缀，「1 账户 1 集群」
本地研发	TestContainers + LocalStack (<code>:4566</code>) 模拟 AWS

论文要素	直接可写的内容
依赖方向	web → common + spec; service → common + spec; batch → common + mdm-spec

五、可用难点素材（按适配度排序）

- 1. ① 一份 proto 契约驱动三语言 ★★ ★ —— 服务间通信、契约一致性
- 2. ② protobuf 版本冲突致启动失败 ★★ ★ —— 服务治理、依赖漂移
- 3. ⑥ Serverless 数据同步与最终一致性 ★★ ★ —— 分布式数据一致性
- 4. ③ AWS SDK v1/v2 并存 ★★ —— 渐进演进（也可放“挑战”类题目）
- 5. ⑨ 分布式会话 Redis Cluster ★★ —— 有状态服务的无状态化

六、踩坑提醒

坑	后果	正确做法
只写「我们用了微服务」	判定为背概念	必须写 拆分依据 ，且依据要来自业务特征
大谈 CAP、BASE 理论	空洞	用一个具体场景说明最终一致性（如 DynamoDB Stream → Lambda → MySQL）
把模块当微服务数	逻辑矛盾	8 个 Gradle 模块 ≠ 8 个微服务，说清「部署单元」与「代码模块」的区别
不提代价	缺乏架构思维	明确写出微服务带来的成本：分布式调试难、跨服务事务缺失、运维复杂度上升
忽略数据边界	关键失分点	服务拆分必须谈数据拆分，本项目「混合存储」是绝佳论据
说的是「服务注册发现」但项目没用	编造风险	本项目未用服务注册中心（gRPC 直连 endpoint + TLS）， 不要编造 Eureka/Nacos

⚠️ 最后一条最重要：BASE HOME 的 gRPC 调用是**配置化 endpoint 直连**（`tls://dev-base-home.hisgroup.app:6565`），并非通过注册中心发现。论文中若写「我们使用 Nacos 做服务发现」，一旦被追问细节就会露馅。诚实写「服务地址通过配置中心化下发，配合 TLS 保证传输安全」同样成立且更真实。

配套：00-项目素材库.md · 06-系统集成与gRPC.md（契约部分可互相复用）